

Horner Evaluation with Integer Overflow Checks

LeanExe library PoC

September 2026

1 Task and interface

`eval` evaluates $p(x) = \sum_{i=0}^{n-1} c_i x^i$ for nonnegative integer coefficients and argument. Each coefficient and x are represented by `UInt64`. The array stores coefficients in ascending degree order. An empty array represents zero. Trailing zero coefficients are accepted.

`LeanExe.Lib.Polynomial.eval` returns `some value` when every Horner intermediate fits in a word and `none` when an intermediate would exceed $M = 2^{64} - 1$. The intended successful result is exact integer evaluation. LeanExe compiles the Lean function and its callers to WebAssembly (WASM). The examples run that WASM in Wasmtime.

2 Verification status

Lean checks the source types. Execution tests pass for selected inputs: WASM and native Lean results agree with an arbitrary-precision sum-of-powers reference, including overflow cases. Formal proofs of polynomial correctness, overflow detection, termination, and the behavior of the generated WASM and clients remain deferred.

3 Recurrence and overflow check

Horner's recurrence processes coefficients from highest degree to lowest, updating $v \leftarrow vx + c_i$. The NIST Digital Library of Mathematical Functions states the recurrence and its evaluation identity [1]. This implementation starts at $v = 0$ and processes each stored coefficient.

For $x > 0$, the condition $vx + c_i \leq M$ is equivalent to

$$v \leq \left\lfloor \frac{M - c_i}{x} \right\rfloor.$$

The check precedes multiplication and addition. Subtraction is safe because $c_i \leq M$. At $x = 0$, each update returns the coefficient, so the check can be skipped. The loop reads the borrowed input array and carries an index and one accumulated value.

```
namespace LeanExe.Lib.Polynomial
```

```
def eval (coefficients : Array UInt64) (x : UInt64) : Option UInt64 := Id.run do
  let mut value : UInt64 := 0
  let mut index := coefficients.size
  while index > 0 do
    index := index - 1
    let coefficient := coefficients[index]!
```

```

    if x != 0 && value > (18446744073709551615 - coefficient) / x then
      return none
    value := value * x + coefficient
    return some value
end LeanExe.Lib.Polynomial

```

The listing comes from `Basic.lean`. For coefficients $[1, 3, 2]$ and $x = 2$, the accumulated values are 2, 7, and 15. The implementation performs n multiply-add updates on a successful evaluation and at most n divisions for the checks. An overflow causes an early return.

4 Clients and commands

The polynomial client converts the optional result to an explicit status and value. The polynomial-ratio client evaluates two arrays, checks the denominator, and reduces the resulting fraction through GCD. The commands run from the repository root after following the setup guide.

```

tools/seminum polynomial 2 1,3,2
15
tools/seminum polynomial 99 '[]'
0
tools/seminum ratio 2 1,3,2 2,2
5/2
tools/seminum polynomial 2 18446744073709551615,1
seminum: arithmetic overflow

```

The last command writes its diagnostic to standard error and exits with status 2. Its exact result would be $M + 2$. Decimal parsing, compilation, and invocation belong to the command driver. Polynomial arithmetic executes in the generated WASM.

5 Tests

From the repository root, `node test/seminum.js` runs the comparisons described above. Cases include empty and constant polynomials, zero arguments, maximum words, exact boundary results, and overflow. Ratio tests check successful composition, a zero denominator, and overflow in either polynomial. CLI tests check results and invalid-input diagnostics. The cases are defined in `test/seminum.js` and the native reference driver `test/SeminumNative.lean`.

References

- [1] NIST Digital Library of Mathematical Functions, Section 1.11(i), equations 1.11.1–1.11.4. <https://dlmf.nist.gov/1.11.i>. These equations supply Horner’s recurrence and evaluation identity. The integer overflow check and optional-result API are choices of this implementation.